

Teacher AI Platform — Full Winning Implementation Plan

Document Type: Assignment Execution Blueprint
Assignment: AI Engineer Assignment — Teacher AI Platform
Status: ACTIVE — Primary Focus
Version: v2.0 (All 10 Stages + All 6 Bonus Features Included)

1. Executive Summary

This document is the **complete, execution-ready plan** to design and build a production-grade AI system that converts raw educational documents into structured, classroom-ready **Teacher Knowledge Packages (TKP)**.

The system processes uploaded documents through **10 sequential AI pipeline stages** spanning document intelligence, pedagogical planning, assessment generation, and validation, and outputs a master `TeacherKnowledgePackage.json` alongside human-readable PDFs and a browsable UI.

All **6 bonus features** are explicitly designed into this system:

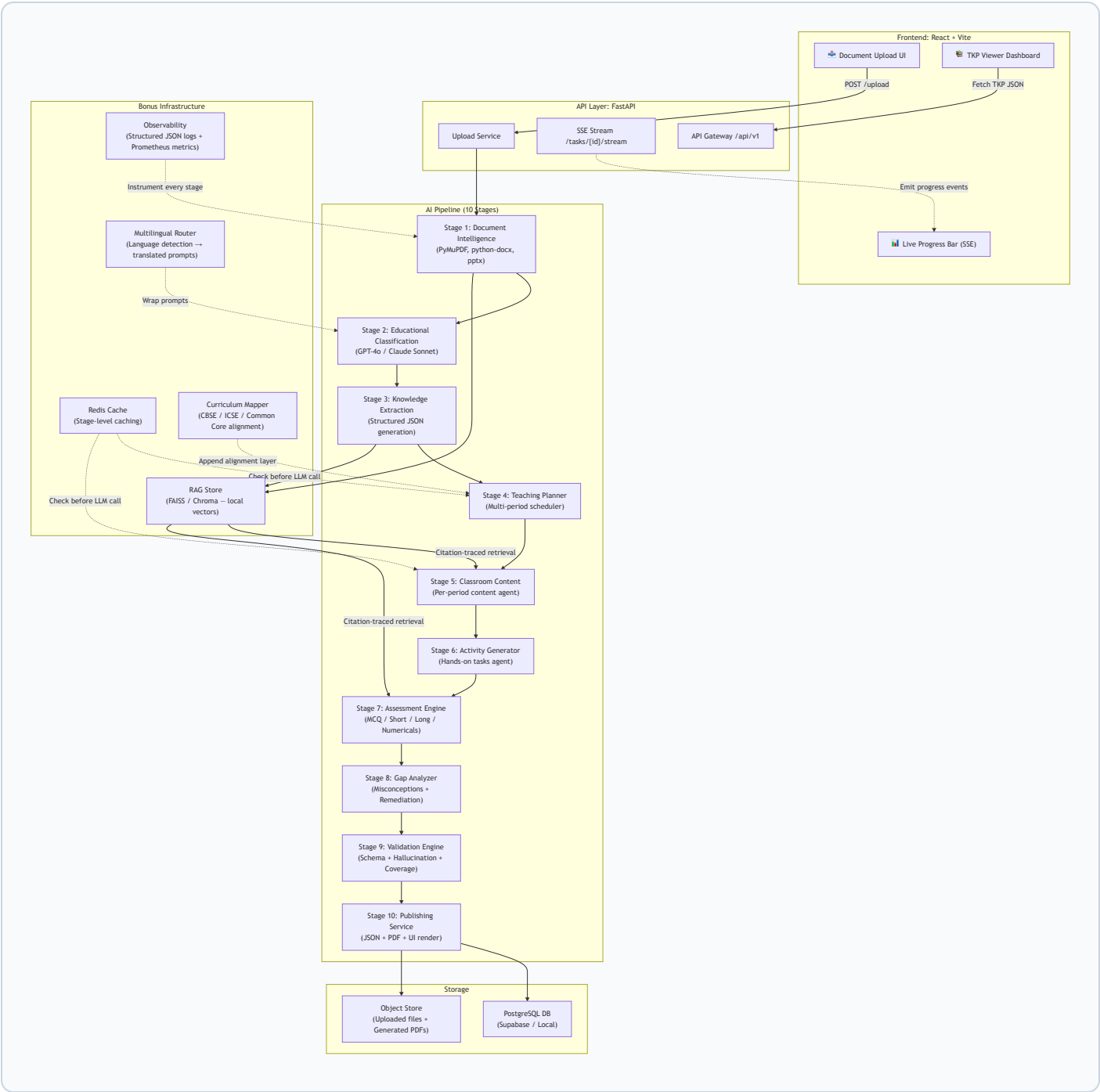
-  Multi-Agent Orchestration
 -  RAG & Traceability
 -  Curriculum Alignment (CBSE, ICSE, Common Core)
 -  Performance Optimization (batching, caching, parallel generation)
 -  Observability (logs, metrics, tracing, retry mechanisms)
 -  Multilingual Support
-

2. Evaluation-Aligned Strategy

Before writing a line of code, we align every architectural decision to the grading rubric:

Evaluation Category	Weight	Our Strategy
Content Generation & Versatility	25%	Multi-domain prompting (STEM formulas + Humanities narratives). LLM outputs validated and schema-strict.
Educational Understanding	20%	Bloom's Taxonomy-aligned objective extraction. Subject-aware classification pipeline.
Teaching Planning	20%	Intelligent multi-period scheduler with 40-minute period templates and adaptive sequencing.
Document Intelligence	15%	Layout-preserving parsing using PyMuPDF + python-docx + python-pptx with fallback OCR.
Engineering & Architecture	15%	Modular FastAPI microservices, SSE progress streaming, Pydantic schema validation.
Documentation & Demo	5%	Auto-generated README, Mermaid architecture diagrams, public GitHub, Render/Vercel deployment.

3. System Architecture



4. Technology Stack

Component	Technology	Why
Backend Framework	FastAPI (Python 3.11)	Async-native, built-in SSE support, excellent AI library ecosystem
LLM Provider	OpenAI GPT-4o / Anthropic Claude Sonnet	Best structured JSON generation; fallback routing between providers
Document Parsing	PyMuPDF, python-docx, python-pptx	Handles PDF, DOCX, PPTX natively; preserves headings, tables, equations
Vector / RAG Store	FAISS (local) or Chroma	Lightweight, zero-cloud dependencies, enables citation-traceable RAG
Database	PostgreSQL via Supabase	Free-tier hosted, reliable, stores task metadata and final TKP JSON
Cache	Redis (Upstash free tier)	Stage-level deduplication: same document ≠ re-run same stage
PDF Generation	WeasyPrint + Jinja2 HTML templates	Renders professional lesson plan books from TKP JSON
Frontend	React 18 + Vite + Vanilla CSS	Handles SSE progress stream, renders TKP in a tabbed UI
Deployment	Render (backend) + Vercel (frontend)	Free-tier, fast CI/CD from GitHub, zero config HTTPS
Observability	Python logging (JSON-L) + optional Prometheus	Structured logs per stage with retry counters, latency, token usage

5. Detailed Stage Specifications

Stage 1 — Document Intelligence

Goal: Parse uploaded files while **preserving structural hierarchy**.

Supported formats: PDF, DOCX, PPTX, TXT

Parser Strategy:

- PDF: `PyMuPDF (fitz)` — extract text block by block, detect headings from font size/weight, extract tables as markdown, detect figure captions.
- DOCX: `python-docx` — walk paragraph styles (Heading 1/2/3, Normal) to rebuild hierarchy.

- **PPTX:** `python-pptx` — extract slide title, body text, speaker notes per slide.
- **Equations:** Detect LaTeX-like patterns ($\frac{\{ \}}{\{ \}}$, $\sum$, $\int$) and preserve raw strings.

Output Structure:

```
{
  "document_id": "uuid",
  "pages": 42,
  "sections": [
    {
      "level": 1,
      "heading": "Laws of Motion",
      "content": "Newton's laws describe the relationship between...",
      "tables": [],
      "figures": [],
      "equations": ["F = ma", "F = dp/dt"]
    }
  ],
  "raw_text": "..."
}
```

Stage 2 — Educational Classification

Goal: Identify the document's pedagogical context.

Method: Single structured LLM call using JSON mode.

Prompt Strategy:

"Given the following extracted text from a document, classify it as an educational resource. Output exactly the following fields: subject, grade_level, difficulty, topic, chapter, category (STEM/Humanities/Social Science/Commerce), and language."

Output:

```
{
  "subject": "Physics",
  "grade_level": "Class 11",
  "difficulty": "Intermediate",
  "topic": "Mechanics",
  "chapter": "Newton's Laws of Motion",
  "category": "STEM",
  "language": "English"
}
```

Stage 3 — Knowledge Extraction

Goal: Extract all teachable knowledge units from the parsed document.

Method: Multi-pass structured LLM extraction using Pydantic models.

Pass 1 — Objectives & Prerequisites:

Extract 5–8 learning objectives using Bloom's Taxonomy verbs. Extract prerequisite concepts a student must know before studying this material.

Pass 2 — Concept Extraction:

For each major concept in the document, extract: name, definition, formulae (as LaTeX strings), real-world examples, applications, and common student misconceptions.

Pass 3 — Keyword Mapping:

Extract a glossary of key terms and their one-sentence definitions.

Stage 4 — Teaching Planner

Goal: Design a sequential, multi-period teaching strategy.

Algorithm:

1. Count the number of objectives from Stage 3.
2. Group objectives into **clusters** (typically 2–3 objectives per period).
3. For each period, define: period number, duration (default 40 min), objectives covered, entry prerequisite check, sub-topics in order.
4. Ensure first period = foundations, last period = synthesis/application.

Curriculum Alignment (Bonus):

- If `curriculum_board` is specified (CBSE/ICSE/Common Core), append board-specific learning outcome codes to each period.

Output:

```
{
  "total_periods": 5,
  "periods": [
    {
      "period_number": 1,
      "duration_minutes": 40,
      "objectives_covered": ["OBJ-1", "OBJ-2"],
      "subtopics": ["Concept of force", "Inertia and mass"],
      "curriculum_alignment": {
        "board": "CBSE",
        "outcome_codes": ["XI-PHY-L1", "XI-PHY-L2"]
      }
    }
  ]
}
```

Stage 5 — Classroom Content Generation

Goal: Generate full, detailed teaching material per period.

This is the core stage — the primary driver of the 25% Content Generation score.

Per-period generation (parallel across all periods):

Component	Description
Entry Ticket	2-3 warm-up questions connecting to prior knowledge
Teacher Script	Full narrative: introduction → explanation → worked examples → Q&A prompts
Blackboard Notes	Clean summary bullets and diagrams/equations the teacher draws
Classroom Activities	1-2 mid-period engagement activities
Checkpoint Questions	Formative check: 2 quick conceptual questions mid-period
Exit Ticket	1 synthesis question with sample answer
Homework	3–5 graduated practice problems
Mentor Moment	A 2-minute motivational anecdote, historical context, or "aha" story

RAG Integration (Bonus): Before generating each component, retrieve the top-3 most relevant chunks from the Stage 1 vector store. Inject them into the prompt and record chunk IDs as `sources[]` for traceability.

Stage 6 — Activity Generation

Goal: Design diverse, hands-on activities beyond standard lectures.

Activity types generated:

- Laboratory Experiment
- Thought Experiment / Socratic Discussion
- Role Play / Debate
- Group Investigation
- Multimedia Demonstration

Per-activity schema:

```
{
  "title": "Newton's Cart Experiment",
  "type": "Laboratory Experiment",
  "duration_minutes": 20,
  "materials": ["Dynamic cart", "Force sensor", "Pulley", "Weights"],
  "teacher_instructions": "Set up the cart on a frictionless track...",
  "student_worksheet": "Record force and measure resulting acceleration...",
  "success_criteria": "Students produce a linear F vs. a graph and calculate mass from slope.",
  "safety_notes": "Ensure weights are secured before releasing the cart."
}
```

Stage 7 — Assessment Generation

Goal: Create a full, multi-format assessment bank.

Assessment types:

- Multiple Choice Questions (MCQs) — 5 per period
- Short Answer Questions — 2 per period
- Long Answer / Essay — 1 per chapter
- Numerical Problems (STEM subjects only) — 3 per period

Each question includes:

- Question text
- Answer key (with full working for numericals)
- Bloom's Taxonomy level tag (Remember/Understand/Apply/Analyze/Evaluate/Create)
- Rubric (for short/long answers)
- `source_chunks[]` for RAG traceability

Stage 8 — Learning Gap Analysis

Goal: Proactively identify and remediate student misconceptions.

Method: LLM prompted with the extracted concepts, known pedagogical literature patterns, and the content generated in Stage 5.

Per-misconception schema:

```
{
  "misconception": "A moving object requires a constant force to keep moving.",
  "severity": "High",
  "affected_objective": "OBJ-2",
  "diagnostic_question": "If you push a hockey puck on ice and let go, what happens?",
  "expected_wrong_answer": "It slows down because there's no force pushing it.",
  "remedial_action": "Use the air track demo to show frictionless motion. Re-teach Newton's First L",
  "remedial_resources": ["Air track video demo", "Worksheet: Friction vs. no friction scenarios"]
}
```

Stage 9 — Validation Engine

Goal: Automated quality assurance before publishing.

Three validation layers:

Layer 1 — Schema Validation:

- Use `Pydantic` to validate the assembled TKP JSON against the master schema.
- If fields are missing or malformed → re-trigger just that stage (not the full pipeline).

Layer 2 — Coverage Check:

- Cross-reference: every Objective from Stage 3 must appear in ≥ 1 Period (Stage 4) and ≥ 1 Assessment question (Stage 7).
- If any objective is untested → flag and generate 1 compensating question.

Layer 3 — Hallucination Detection:

- For every factual claim in Teacher Scripts (Stage 5), retrieve the top-1 source chunk from the RAG store.
- Calculate cosine similarity. If similarity < 0.65 → flag sentence with `low_confidence: true` for human review.

Stage 10 — Publishing Service

Goal: Produce all final deliverables in consumable formats.

Output 1: `TeacherKnowledgePackage.json` A fully assembled master JSON file combining outputs from all 9 prior stages.

Output 2: PDF Lesson Plan Book Generated using WeasyPrint + Jinja2 HTML templates. Contains:

- Cover page with subject/grade/chapter
- One page per period (Teacher Script + Blackboard Notes)
- Activity instructions

Output 3: PDF Assessment Book

- Student version (questions only, no answers)
- Teacher version (questions + answer keys + rubrics)

Output 4: UI Render The frontend TKP Viewer dynamically renders the JSON into:

- Tabbed navigation (Overview → Periods → Assessments → Activities → Gaps)
- Collapsible period sections
- Syntax-highlighted JSON raw view

6. Bonus Features — Implementation Details

✅ Bonus 1: Multi-Agent Orchestration

Each stage is implemented as an isolated **Agent class** with a single `run(input) → output` interface. The pipeline orchestrator (`PipelineCoordinator`) calls them sequentially, passing typed outputs between agents. This ensures:

- Independent testability of each agent
- Ability to swap LLM providers per agent
- Partial re-run on validation failure (only re-run the failing agent)

```
DocumentIntelligenceAgent
ClassificationAgent
KnowledgeExtractionAgent
PlannerAgent
ContentGenerationAgent ← runs in parallel per period
ActivityAgent
AssessmentAgent
GapAnalysisAgent
ValidationAgent
PublishingAgent
```

✅ Bonus 2: RAG & Traceability

Implementation:

1. After Stage 1, chunk the document text into 512-token overlapping chunks.
2. Embed chunks using `text-embedding-3-small` (OpenAI) or `all-MiniLM-L6-v2` (local via sentence-transformers).
3. Store in FAISS (local) or Chroma.
4. During Stage 5 and Stage 7, retrieve top-3 relevant chunks per generation call.
5. Each generated artifact carries a `sources` array:

```
"sources": [
  {"chunk_id": "ch-42", "text": "Newton's second law states...", "page": 12, "section": "3.2"}
]
```

✅ **Bonus 3: Curriculum Alignment**

Implementation:

- A `curriculum_board` parameter is accepted at upload time (`CBSE` , `ICSE` , `Common Core` , `IB`).
- A static mapping file `curriculum_standards.json` maps subject/grade combinations to board-specific outcome codes.
- The Planner Agent appends these codes to each period's output.
- Final PDFs include a "Curriculum Alignment Table" appendix.

✅ **Bonus 4: Performance Optimization**

Strategy	Implementation
Stage-level Caching	Redis key = <code>hash(document_id + stage_name)</code> . If key exists, return cached result instantly.
Parallel Period Generation	Stage 5 (Classroom Content) is generated for all periods simultaneously using <code>asyncio.gather()</code> .
LLM Batching	Assessment questions for all periods are batched in a single LLM call with a structured list prompt.
Model Tiering	Simple tasks (Stage 2 classification) use <code>gpt-4o-mini</code> ; complex generation (Stage 5 scripts) uses full <code>gpt-4o</code> .

✅ **Bonus 5: Observability**

Implementation using Python's `logging` module with JSON-L output:

Every stage emits structured log entries:

```
{
  "timestamp": "2026-07-31T11:30:00Z",
  "stage": "classroom-content",
  "period": 2,
  "duration_ms": 184
}
```

Retry Mechanism: All LLM calls are wrapped in an `@retry(max_attempts=3, backoff=exponential)` decorator. On repeated failure, the stage marks itself `degraded` and continues the pipeline with a placeholder.

Prometheus Metrics (optional):

- `tkp_stage_duration_seconds{stage="..."}`
- `tkp_llm_tokens_total{model="..."}`
- `tkp_retry_count{stage="..."}`

✅ Bonus 6: Multilingual Support

Implementation:

1. Stage 2 classification detects document language automatically.
2. If `output_language` is specified at upload (e.g., `Hindi`, `French`, `Spanish`), a **Language Router** wraps all generation prompts with:

"Generate the following content in {output_language}. Maintain all technical terminology in English within parentheses."

3. PDF templates are rendered with appropriate fonts (Noto Sans for Hindi/Arabic support).
 4. The TKP JSON includes a `language` field throughout.
-

7. Data Schema: TeacherKnowledgePackage.json

```
{
  "tkp_version": "1.0",
  "document_id": "uuid",
  "generated_at": "ISO-8601 timestamp",
  "metadata": {
    "subject": "Physics",
    "grade_level": "Class 11",
    "difficulty": "Intermediate",
    "topic": "Mechanics",
    "chapter": "Newton's Laws of Motion",
    "category": "STEM",
    "language": "English",
    "output_language": "English",
    "curriculum_board": "CBSE"
  },
  "document_intelligence": {
    "pages": 42,
    "sections_detected": 8,
    "equations_found": 12,
    "tables_found": 3
  },
  "knowledge_extraction": {
    "learning_objectives": [
      { "id": "OBJ-1", "description": "...", "bloom_level": "Apply" }
    ],
    "prerequisites": ["Vector quantities", "Kinematics"],
    "concepts": [
      {
        "name": "Inertia",
        "definition": "...",
        "formulae": [],
        "keywords": ["mass", "resistance", "rest"],
        "examples": ["..."],
        "applications": ["..."],
        "misconceptions": ["..."],
        "sources": [{ "chunk_id": "ch-5", "page": 4 } ]
      }
    ]
  },
  "teaching_plan": {
    "total_periods": 5,
    "period_duration_minutes": 40,
    "periods": [
      {
        "period_number": 1,
        "objectives_covered": ["OBJ-1"],
        "subtopics": ["Concept of Force"],
        "curriculum_alignment": { "board": "CBSE", "codes": ["XI-PHY-L1"] },
        "classroom_content": {
          "entry_ticket": { "questions": ["..."], "purpose": "...",

```

```

    "teacher_script": {"introduction": "...", "key_points": ["..."], "transitions": ["..."]},
    "blackboard_notes": "...",
    "checkpoint_questions": ["...", "..."],
    "activities": [{"title": "...", "type": "Demo", "duration_minutes": 10}],
    "mentor_moment": "...",
    "exit_ticket": {"question": "...", "answer_key": "..."},
    "homework": ["..."]
  }
}
],
"activities": [],
"assessments": [
  {
    "id": "Q-1",
    "period": 1,
    "type": "mcq",
    "bloom_level": "Apply",
    "question": "...",
    "options": {"A": "...", "B": "...", "C": "...", "D": "..."},
    "correct_answer": "C",
    "explanation": "...",
    "rubric": null,
    "sources": [{"chunk_id": "ch-12"}]
  }
],
"learning_gap_analysis": [
  {
    "misconception": "...",
    "severity": "High",
    "affected_objective": "OBJ-2",
    "diagnostic_question": "...",
    "remedial_action": "...",
    "sources": []
  }
],
"validation_report": {
  "schema_valid": true,
  "objectives_coverage": {"covered": 5, "total": 5, "uncovered": []},
  "hallucination_flags": [],
  "overall_confidence": "High"
}
}

```

8. Repository & File Structure

```

teacher-ai-platform/
├── backend/
│   ├── main.py                # FastAPI app entry point
│   ├── requirements.txt
│   ├── config.py              # Environment variables
│   ├── agents/
│   │   ├── base_agent.py      # Abstract base agent
│   │   ├── document_agent.py   # Stage 1: Parsing
│   │   ├── classification_agent.py # Stage 2
│   │   ├── extraction_agent.py  # Stage 3
│   │   ├── planner_agent.py     # Stage 4
│   │   ├── content_agent.py     # Stage 5 (parallel)
│   │   ├── activity_agent.py   # Stage 6
│   │   ├── assessment_agent.py  # Stage 7
│   │   ├── gap_agent.py        # Stage 8
│   │   ├── validation_agent.py  # Stage 9
│   │   └── publishing_agent.py  # Stage 10
│   ├── pipeline/
│   │   ├── coordinator.py      # Orchestrator: runs 10 stages in order
│   │   └── sse_manager.py      # Server-Sent Events progress emitter
│   ├── rag/
│   │   ├── chunker.py          # 512-token overlapping chunker
│   │   ├── embedder.py         # Embedding model wrapper
│   │   └── retriever.py        # FAISS/Chroma vector store access
│   ├── schemas/
│   │   └── tkp_schema.py       # Pydantic TKP master schema
│   ├── templates/
│   │   ├── lesson_plan.html    # Jinja2 PDF template
│   │   └── assessment_book.html # Jinja2 PDF template
│   ├── observability/
│   │   ├── logger.py           # Structured JSON-L logger
│   │   └── metrics.py          # Prometheus metrics (optional)
│   └── utils/
│       ├── cache.py            # Redis cache wrapper
│       ├── retry.py            # Retry decorator
│       └── curriculum_mapper.py # Board alignment lookup
├── frontend/
│   ├── package.json
│   ├── vite.config.js
│   ├── index.html
│   └── src/
│       ├── App.jsx
│       ├── index.css
│       ├── components/
│       │   ├── UploadZone.jsx  # Drag-and-drop upload
│       │   ├── ProgressTracker.jsx # SSE-driven live progress bar
│       │   ├── TKPViewer.jsx   # Tabbed TKP result viewer
│       │   ├── PeriodCard.jsx  # Per-period content display
│       │   └── AssessmentView.jsx # MCQ/Short answer browser

```


Day 1 — Foundation & Parsing (Stages 1-3)

Block	Task	Output
Morning	Scaffold repo: FastAPI skeleton, React+Vite setup, <code>.env</code> config, Docker Compose	Running <code>uvicorn</code> + Vite dev server
Morning	Install all dependencies: PyMuPDF, python-docx, python-pptx, openai, pydantic, weasyprint, faiss-cpu, redis	<code>requirements.txt</code> locked
Afternoon	Stage 1: Document Intelligence agent — PDF/DOCX/PPTX parser, heading detection, equation/table extraction	<code>DocumentIntelligenceAgent</code> returns structured JSON
Afternoon	RAG chunker + FAISS embedder — 512-token overlapping chunker, embed with <code>text-embedding-3-small</code> , store in FAISS	Vector store built from uploaded doc
Evening	Stage 2: Educational Classification agent — single structured LLM call, subject/grade/difficulty/language output	<code>ClassificationAgent</code> returning metadata JSON
Evening	Stage 3: Knowledge Extraction agent — multi-pass: objectives (Bloom's), concepts, formulae, misconceptions	<code>KnowledgeExtractionAgent</code> returning full extraction JSON

Day 2 — AI Pipeline Stages 4–8 + Validation

Block	Task	Output
Morning	Stage 4: Teaching Planner agent — multi-period scheduler, CBSE/ICSE/Common Core alignment layer	<code>PlannerAgent</code> returning period plan JSON
Morning	Stage 5: Classroom Content agent — entry ticket, teacher script, blackboard notes, exit ticket, homework, mentor moment. Run parallel across all periods using <code>asyncio.gather()</code>	<code>ContentAgent</code> per period; all periods generated simultaneously
Afternoon	Stage 6: Activity Generation agent — lab experiments, role-plays, demonstrations per period	<code>ActivityAgent</code> returning activity JSON per period
Afternoon	Stage 7: Assessment Engine — MCQs, short/long answer, numericals with answer keys and Bloom's level tags	<code>AssessmentAgent</code> returning full question bank
Evening	Stage 8: Gap Analysis agent — misconceptions, severity, diagnostic questions, remedial actions	<code>GapAnalysisAgent</code> output
Evening	Stage 9: Validation Engine — Pydantic schema check, objective coverage check, RAG hallucination cosine-similarity check	<code>ValidationAgent</code> with pass/fail report

Day 3 — Publishing, Streaming, Frontend + All Bonus Features

Block	Task	Output
Morning	Stage 10: Publishing agent — assemble master <code>TeacherKnowledgePackage.json</code> ; Jinja2 + WeasyPrint PDF generation for Lesson Plan Book and Assessment Book	Downloadable PDFs + master JSON
Morning	SSE Pipeline Coordinator — <code>PipelineCoordinator</code> calling all 10 agents in sequence, emitting SSE events per stage with <code>progress %</code> and <code>description</code>	<code>/tasks/{id}/stream</code> SSE endpoint live
Afternoon	Frontend: Upload Zone (drag-and-drop), ProgressTracker (SSE-driven live bar), TKP Viewer (tabbed: Overview / Periods / Assessments / Activities / Gaps), PDF download buttons	Full working UI rendering live TKP output
Afternoon	Bonus — RAG Traceability: Wire retriever into Stage 5 and Stage 7 prompts; attach <code>sources[]</code> array to each generated artifact	Citation-traceable lesson scripts and questions
Evening	Bonus — Curriculum Alignment: <code>curriculum_standards.json</code> lookup + board code appended to each period; PDF appendix table	CBSE/ICSE/Common Core codes in TKP JSON
Evening	Bonus — Performance: Redis cache wrapper (<code>hash(doc_id + stage)</code> key), model tiering (<code>gpt-4o-mini</code> for Stage 2, <code>gpt-4o</code> for Stage 5), <code>asyncio.gather()</code> for parallel period generation	Cache hits skip LLM calls; cost reduced
Evening	Bonus — Observability: JSON-L structured logger per stage (timestamp, duration_ms, tokens_used, retry_count); <code>@retry(max=3, backoff=exponential)</code> decorator on all LLM calls	Logs emitted to console + file; retries auto-handled
Evening	Bonus — Multilingual: Language router wrapping all generation prompts with <code>output_language</code> param; Noto Sans font in PDF templates for non-Latin scripts	Hindi/French/Spanish lesson plans generated

Day 4 — Sample Outputs, Deployment, README, Polish

Block	Task	Output
Morning	Generate Sample 1: Physics Class 11 — Newton's Laws of Motion (STEM: equations, numericals, lab activity)	<code>/samples/sample_tkp_physics.json</code>
Morning	Generate Sample 2: History Class 10 — The French Revolution (Humanities: narratives, essays, role-play, multilingual French output)	<code>/samples/sample_tkp_history.json</code>
Afternoon	Deploy backend to Render (FastAPI + Redis env vars set) — test all endpoints live	<code>https://tkp-api.onrender.com</code> live
Afternoon	Deploy frontend to Vercel (set <code>VITE_API_URL</code> pointing to Render backend) — test full upload → stream → view flow	<code>https://tkp-app.vercel.app</code> live
Evening	Write <code>README.md</code> — local setup instructions, architecture diagram (Mermaid), AI orchestration explanation, sample output links	Submission-ready README
Evening	Final end-to-end smoke test: upload Physics PDF → watch SSE progress → view TKP → download PDFs → verify JSON schema	All submission checklist items 

11. Sample Output Strategy

To maximally impress evaluators, generate **two contrasting sample TKPs**:

Sample 1: STEM — Physics Class 11

- Chapter: Newton's Laws of Motion
- Demonstrates: formula extraction ($F=ma$), numerical assessments, lab activity generation, equation preservation in blackboard notes.

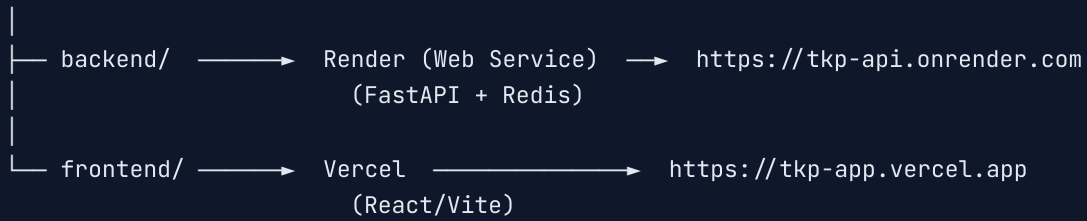
Sample 2: Humanities — History Class 10

- Chapter: The French Revolution
- Demonstrates: subjective narrative extraction, cause-effect learning objectives (Bloom's Analyze level), essay-style assessments with rubrics, timeline-based teaching planner, role-play activities, and multilingual output (e.g., French output language option).

This directly responds to the evaluation note: *"A major part of the evaluation will assess how well your system handles a diverse range of subjects."*

12. Deployment Architecture

GitHub Repository



Environment Variables Required:

```
OPENAI_API_KEY=sk- ...  
DATABASE_URL=postgresql:// ...  
REDIS_URL=redis:// ...
```

13. Submission Checklist

- ☐ Live prototype URL (Vercel + Render)
- ☐ GitHub repository (public) with complete source
- ☐ README with: local setup, architecture diagram, AI orchestration explanation
- ☐ `/samples` folder with 2× `TeacherKnowledgePackage.json` files (Physics + History)
- ☐ All 10 pipeline stages functional end-to-end
- ☐ All 6 bonus features implemented and documented
- ☐ SSE progress streaming working in the frontend
- ☐ PDF export working (Lesson Plan Book + Assessment Book)
- ☐ Multi-subject demo recorded (STEM + Humanities inputs)